ENGINEERING

McMaster University

Lab 6

ELECENG 3EY4

Alaa Abdelsalam – abdela74 – 400396373

Tina Ismail – ismait1 – 400299939

Marisa Patel – patem156 – 400393635

March 7, 2024

Table of Contents

Group Contributions

Marisa: Activities 1-35

Alaa: Activities 1-35

Tina: Activities 1-35

Activity 1

In activity 1, we began by installing the simulator software stack by downloading and building the F1TENTH Simulator from the source code provided on the GitHub repository. In this repository, a set of instructions were provided to follow to download and build the provided code. To clone the repo with the simulator and starter code into the catkin workspace, we used the commands “cd ~/catkin_ws/src” and “git clone https://github.com/f1tenth/f1tenth_simulator.git.” To build this, we ran the “catkin_make” command.

Activity 2

In activity 2, we replaced the existing “params.yaml” with another file posted on Avenue, and then we examined the file.

“rosdc f1tenth_simulator/”: This command navigates to the directory where the “params.yaml” file is located. “roscd” is used to change the directory to the package.

“sudo gedit params.yam1”: This command opens a new file called “params.yam1” in text editor, to view and modify the contents in the file.

Examining the file “params.tam1,” we observed that it contains simulation parameters, physical dimensions, speed limits, and the steering angle. It also includes Lidar parameters, joystick and keyboard control configurations, obstacle parameters, topic names for communication, and VESC control transformation.

Activity 3

In activity 3, we set the values of the parameters “update_pose_rate” and “scan_beams” in the “params.yaml” file so they reflect the specifications of the Lidar scanner used in the actual vehicle.

$update\_pose\_rate = 0.001\ (default)$

$scan\_beams = 1080\ (default)$

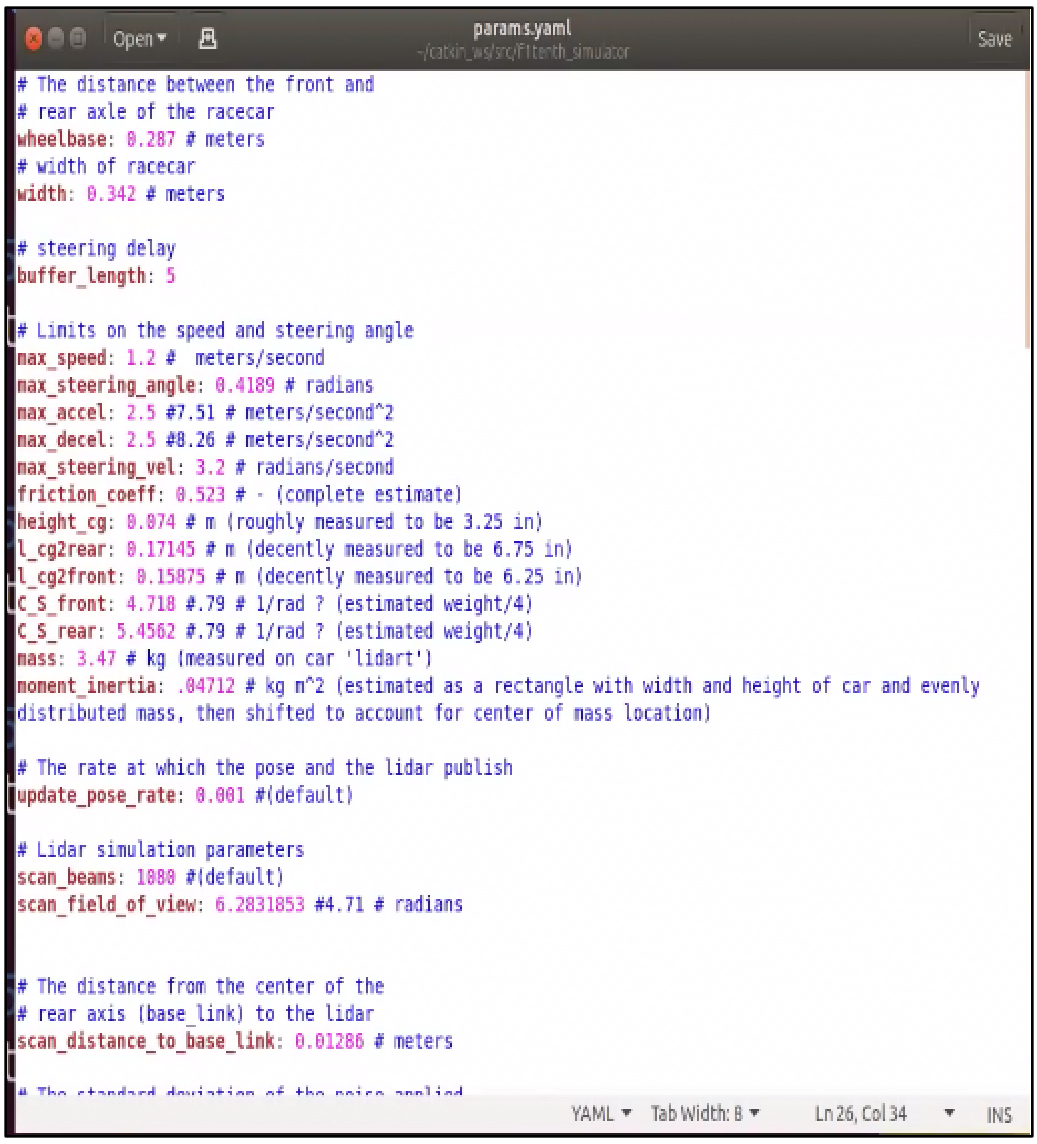

Figure 1. Edited "update_pose_rate" and "scan_beams" Values in "params.yaml" File

Activity 4

In activity 4, we examined the contents of the "simulator.launch" file, which is under the "/launch" directory in the simulator package. This file is a ROS Launch file. This file is used to launch various nodes for the F1TENTH simulator. It launches the "joy_node," which listens to the messages from the joystick. It also launches the "map_server" node with a specific map located in the maps folder. It launches the race car model using the "racecar_model.launch" launch file. Using parameters from the "params.yaml" file, it launches the "f1tenth_simulator" node. Using parameters from the "params.yaml" file, it also launches various nodes including "mux_controller," "behavior_controller," "random_walker," and "keyboard." It also launches "rviz" with a configuration file called "simulator.rviz."

Activity 5

A ROS system consists of numerous independent nodes. A node in ROS is an executable file, such as written in Python or C++, that performs a certain task. A node in ROS can also be referred to as a process that performs computation. Nodes also communicate with various other nodes through topics. Nodes in ROS send and receive data using a publish/subscribe messaging model. The publisher object of a ROS node will publish the data in the form of a message over a topic. Then, the data is received or subscribes through the subscriber object of the ROS node. They also communicate with alternate nodes through services.

Activity 6

The role of a Launch file in ROS is to execute and manage multiple ROS nodes that we configure with specific parameters. It allows users to specify all the nodes, parameters, and configurations in one file. In addition, launch files can specify which nodes to run, setup node namespaces, as well as remapping.

Using a launch file is more efficient and with higher advantages over running node individually with the “rosrun” command. With “rosrun,” each mode needs to be started separately, and manually specifying the parameters. In contrast, launch files enable the initiation of multiple nodes, along with their respective parameters and configurations, all in a single command. Using launch files simplifies the management of complex ROS systems by providing structural approach to node creation and reducing the chance of errors and enhancing the organization and scalability of the overall system.

Command: “rosrun package node parameter:=value”

Since “rosrun” requires manually specifying each node and its parameters, it is not efficient to use for larger systems with multiple nodes and dependencies. It is time-consuming and error-prone to use the “rosrun” command for larger data, and it does not offer the same level of flexibility as launch files do. Therefore, “rosrun” is less suitable for managing complex ROS systems.

Activity 7

In activity 7, we attempted to launch one of the nodes from the list of nodes in the launch file. This was done using the “rosrun” command. We attempted to run the command “rosrun,” for the joystick node, which is the “joy_node.”

Activity 8

The “simulator.launch” file launched various nodes for the operation of the F1TENTH simulator where each node performs a specific function. These nodes include “joy_node,” “map_server,” “f1tenth_simulator,” “mux_controller,” “bahavior_controller,” “random_walker,” “keyboard,” “rviz,” and other new planner nodes.

These nodes facilitate the F1TENTH simulation race car within a virtual setting, enabling manual control and automated navigation. While also supporting the development and testing of new planning algorithms.

“joy_node”: This node monitors joystick messages to enable manual control. This corresponds to the “Joystick” block in Figure 1 of the lab manual.

“map_server”: This node provides a map sourced from the designated maps folder, which is essential for navigation and localization tasks.

“f1tenth_simulator”: This node oversees the fundamental simulation tasks, encompassing vehicle dynamics, sensor emulation, and processing control directives. This corresponds to the “Simulator” block in Figure 1 of the lab manual.

“mux_controller”: This node manages the selection of input sources such as the joystick, keyboard, or planner that govern the simulator, guided by a selector signal. This corresponds to the “Mux” block in Figure 1 of the lab manual.

“bahavior_controller”: This node determines the operational mode (manual or automatic) by analyzing user input alongside sensor data. This corresponds to the “Behavior Controller” block in Figure 1 of the lab manual.

“random_walker” and “keyboard”: These nodes both offer diverse approaches for generating control commands to operate the simulator. The “random_walker” node implements a behavior to the simulated race car. This corresponds to the “Random Driver” block in Figure 1 of the lab manual. The “keyboard” node allows for control of the simulated race car using keyboard inputs. This corresponds to the “Keyboard” block in Figure 1 of the lab manual.

“rviz”: This node launches the rviz visualization tool. It displays the simulation environment and the sensor data from the vehicle in real-time for visualization. This corresponds to the “Simulator” block in Figure 1 of the lab manual.

New planner node: New planner nodes are for creating any new or custom nodes. This corresponds to the “New Planner” block in Figure 1 of the lab manual.

Activity 9

In activity 9, we launched the simulator by executing the command “roslaunch f1tenth_simulator simulator.launch.” This command launches several nodes along with the “rviz” environment. Using the joystick’s LB button, we selected the joystick as the input device to control the vehicle, allowing us to drive the vehicle in the virtual environment with the left and right analog mini-sticks. The left analog mini-stick controls the steering, while the right analog mini-stick controls the speed. The LT button on the joystick enables manual driving with the joystick.

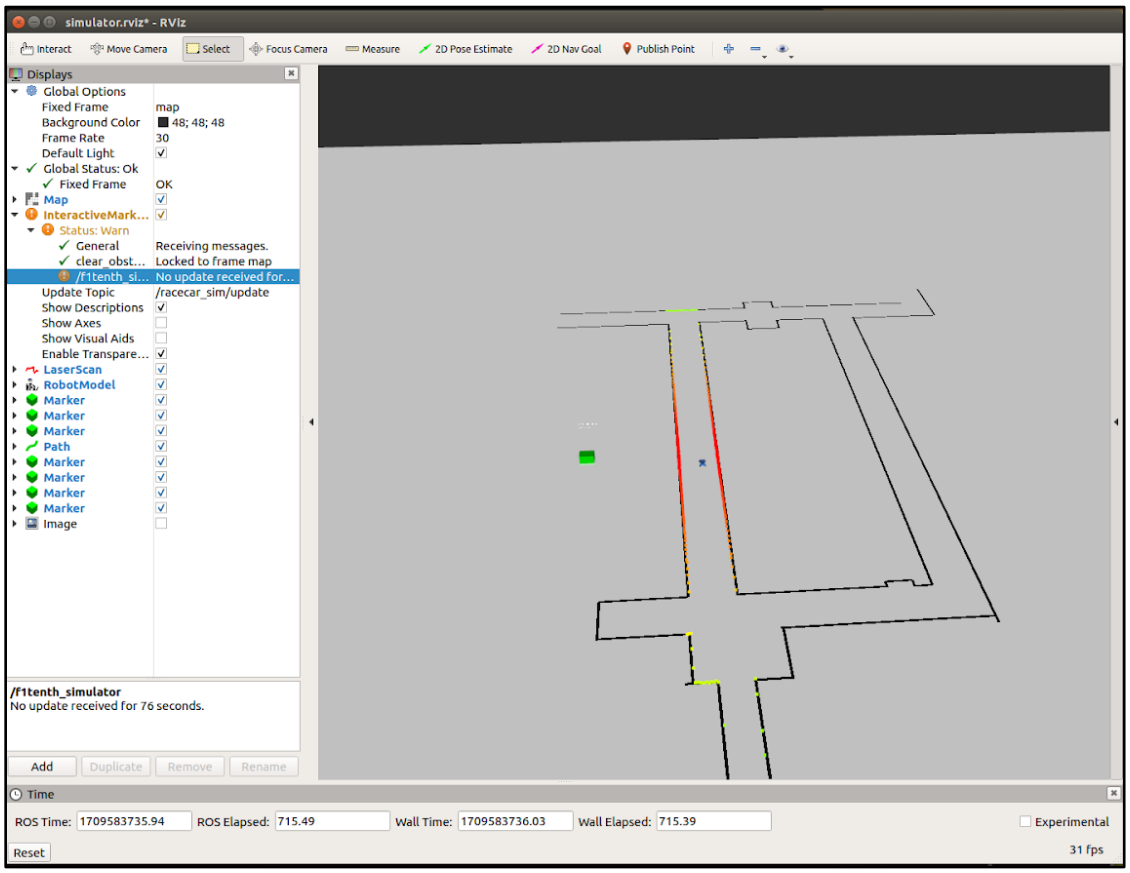

Figure 2. “rviz” Visualization Environment

Activity 10

In activity 10, we ran the “roscore” command to initialize the ROS Master, which is essential for ROS communication. Then, we ran the “rosparam set enable_statistics true” command in a new terminal to enable statistics collection for topics. We ran the simulator again by launching the “simulator.launch” file, which starts a simulation environment with various nodes. To visualize the ROS computation graph, we executed the “rosrun rqt_graph” command to launch the “rqt_graph” shown in Figure 3. This graph displays the nodes currently running, their

communication with topics, and the topic update rates. As the vehicle was driving, we refreshed the graph to see the updated graph. Then, we switched the graph display type to “Nodes/Topics (all)” as shown in Figure 4, to obtain more detailed information about nodes and topics present in the system. Therefore, using “rqt_graph” command, we were able to analyze the communication between different nodes in the ROS system.

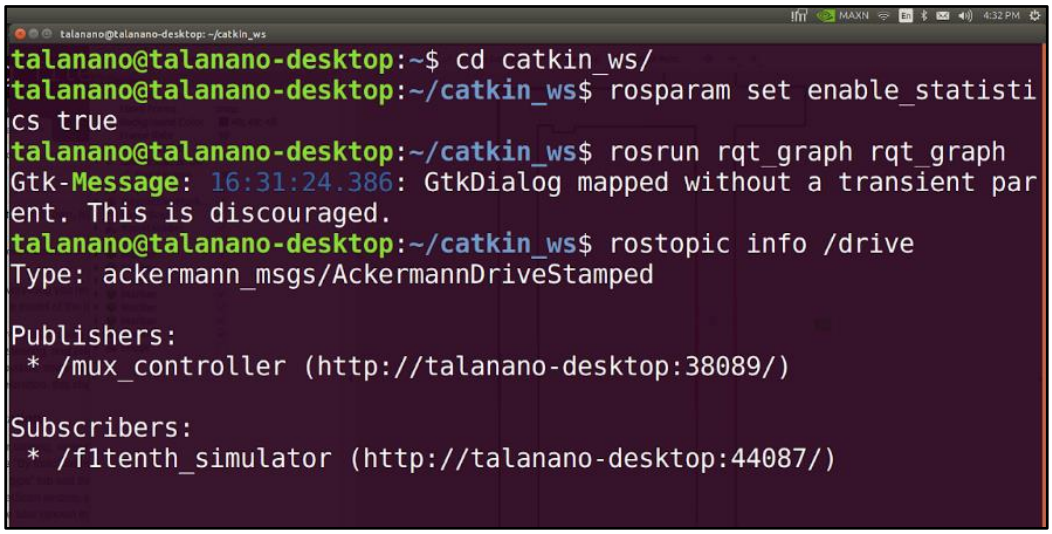

Figure 3. Commands Executed

Activity 11

In ROS, topics are communication channels that facilitates data exchange between different nodes in a system. Some nodes can publish data to topics, and other nodes can subscribe to those topics to receive the data. Topics are highly significant when it comes to data exchange in ROS. Topics facilitate communication between nodes. Topics enable nodes to communicate asynchronously, using its decentralized structure, which is very important for building complex systems. In ROS, topics enable nodes to share information such as sensor data, control commands, status updates, and more.

Activity 12

To visualize the ROS computation graph, we executed the “rosrun rqt_graph” command to launch the “rqt_graph” shown in Figure 3. This graph displays the nodes currently running, their communication with topics, and the topic update rates. As the vehicle was driving, we refreshed the graph to see the updated graph. Then, we switched the graph display type to “Nodes/Topics (all)” as shown in Figure 4, to obtain more detailed information about nodes and topics present in the system.

The node “/f1tenth_simulator” publishes to topics such as “/scan” for lidar data, “/odom” for odometer information, and “/map” for map data. This is indicated by the outward arrows in the computation graph. It subscribes to “/drive” to receive control commands. This is indicated by the incoming or inward arrow in the computation graph.

The update rates of those topics depend on factors such as the processing speed of the node, the rate at which the new data is received, and the complexity of the processed data.

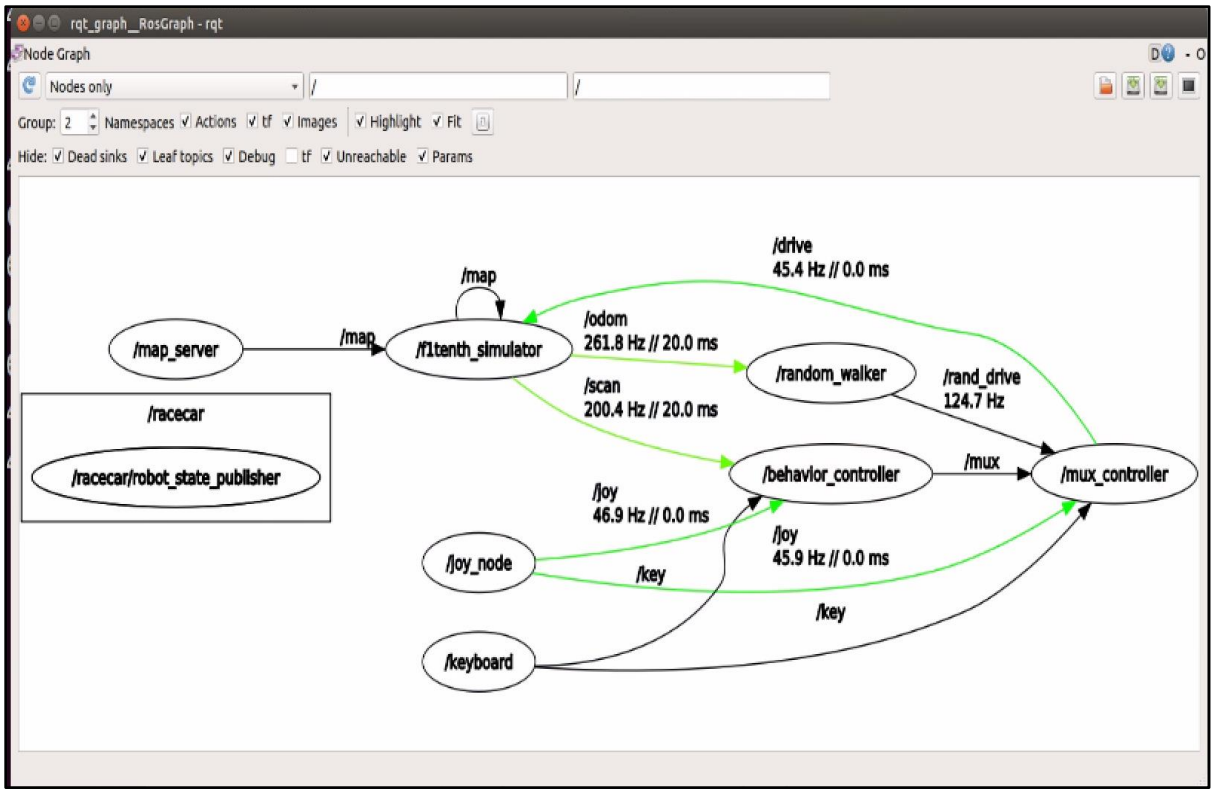

Figure 4. ROS Computation Graph

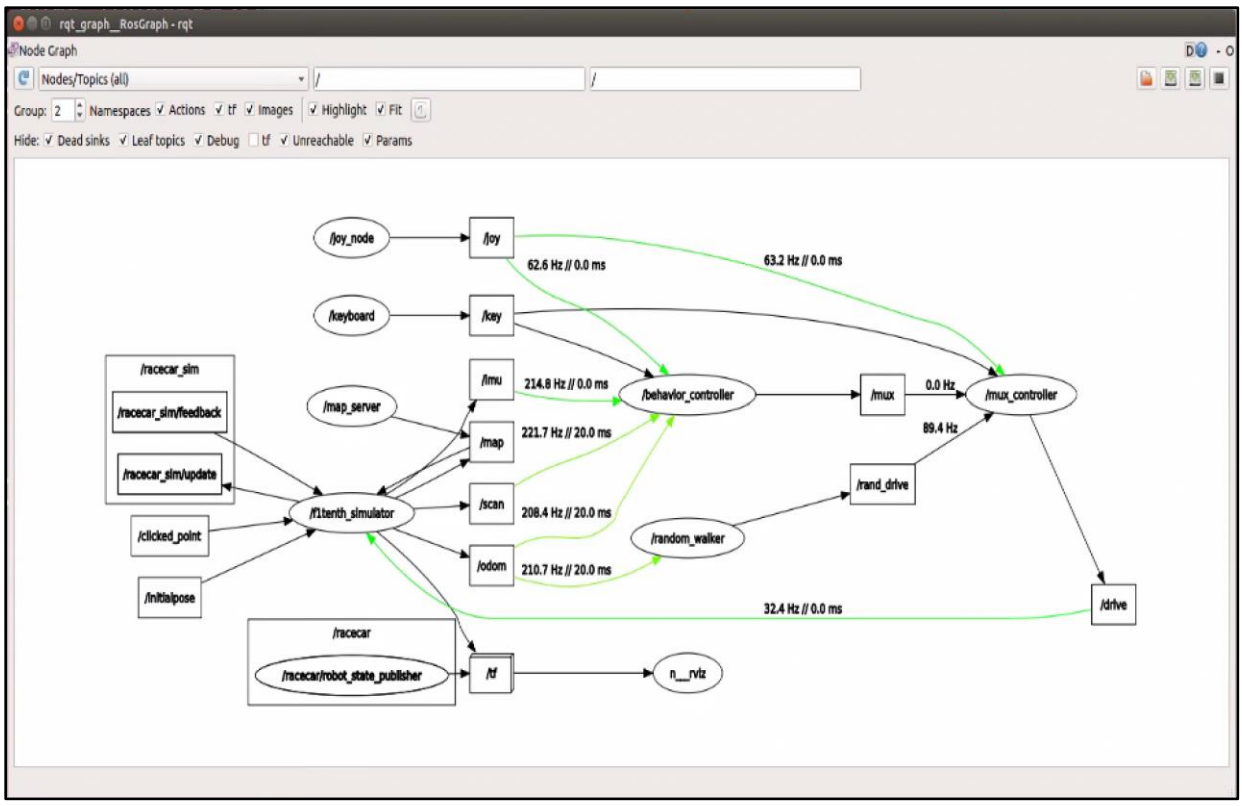

Figure 5. ROS Computation Graph with Additional Details

Activity 13

In activity 13, we executed the “rostopic info /drive” command to obtain information on the “/drive” topic. After executing this command, we observed that the data published on this topic is of “AckermannDrive” type. The provided information tells us that this assumes Ackermann steering is for front-wheel steering. The Ackermann steering command is comprised of a linear velocity command and a vehicle steering command. Since the right and left front wheels are usually at different angles, the commanded angle corresponds to the yaw of the virtual wheel at the center of the front axle. We also learned about the zero steering angle, speed, acceleration, jerk, zero acceleration, and zero jerk.

Activity 14

In activity 14, to echo the data in the “/drive/drive” topic using the “rostopic” command, which is driving the vehicle in the simulator, we used the following steps. First, we used the “roscore” command to start the ROS Master. Then in a new terminal, we ran “rostopic echo /drive/drive.” Then we were able to launch the simulator and drive the vehicle. We can observe the data being echoed in real-time about the velocity, steering angle, and other control commands sent to the vehicle.

Activity 15

The node that publishes to the "/drive” topic is the “mux_controller” or any other nodes that are responsible for processing the input of joystick or autonomous control algorithms. The rate is determined by the specifics of the topic in the computation graph and is dependent upon the frequency at which command messages are generated, based on either user input or decisions made by autonomous algorithms. The updated fields of the “/drive/drive” are the steering angle, steering angle velocity, speed, acceleration, and the jerk as shown in Figure 7. These updated fields are various aspects of vehicle control.

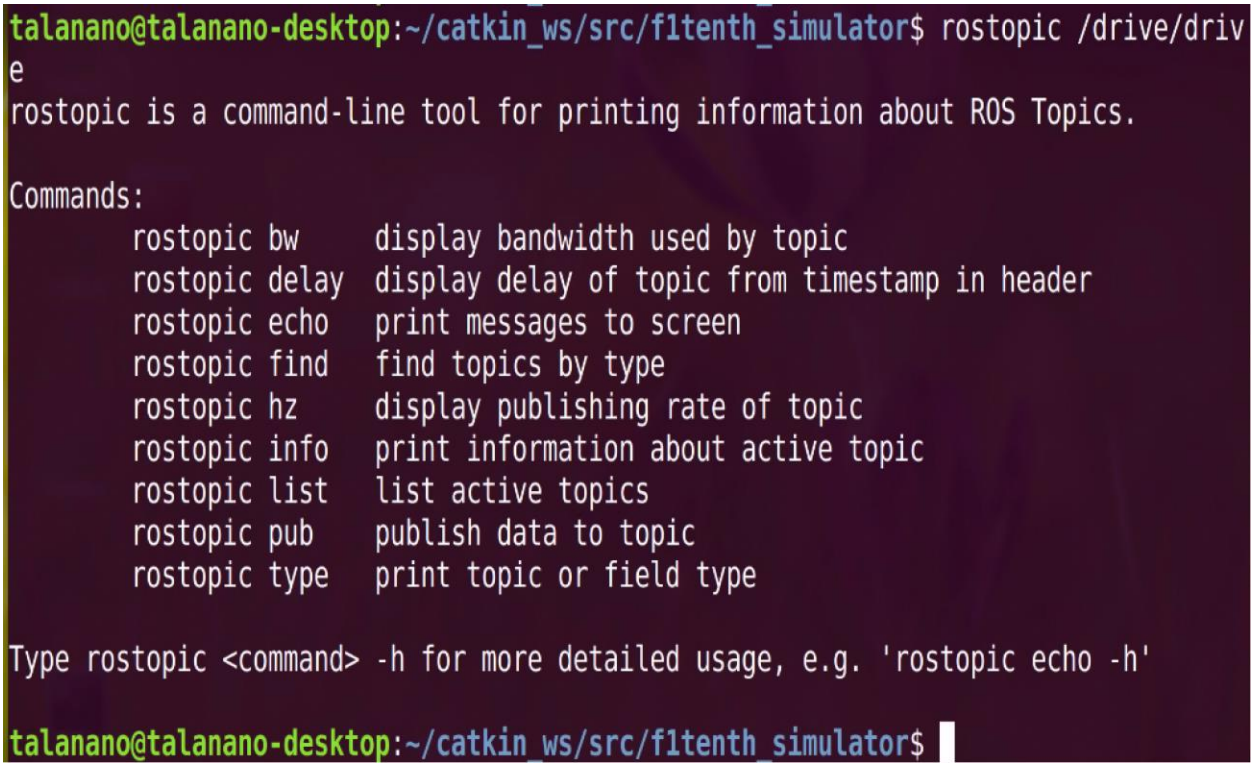

Figure 6. “rostopic /drive/drive”: Command and Output

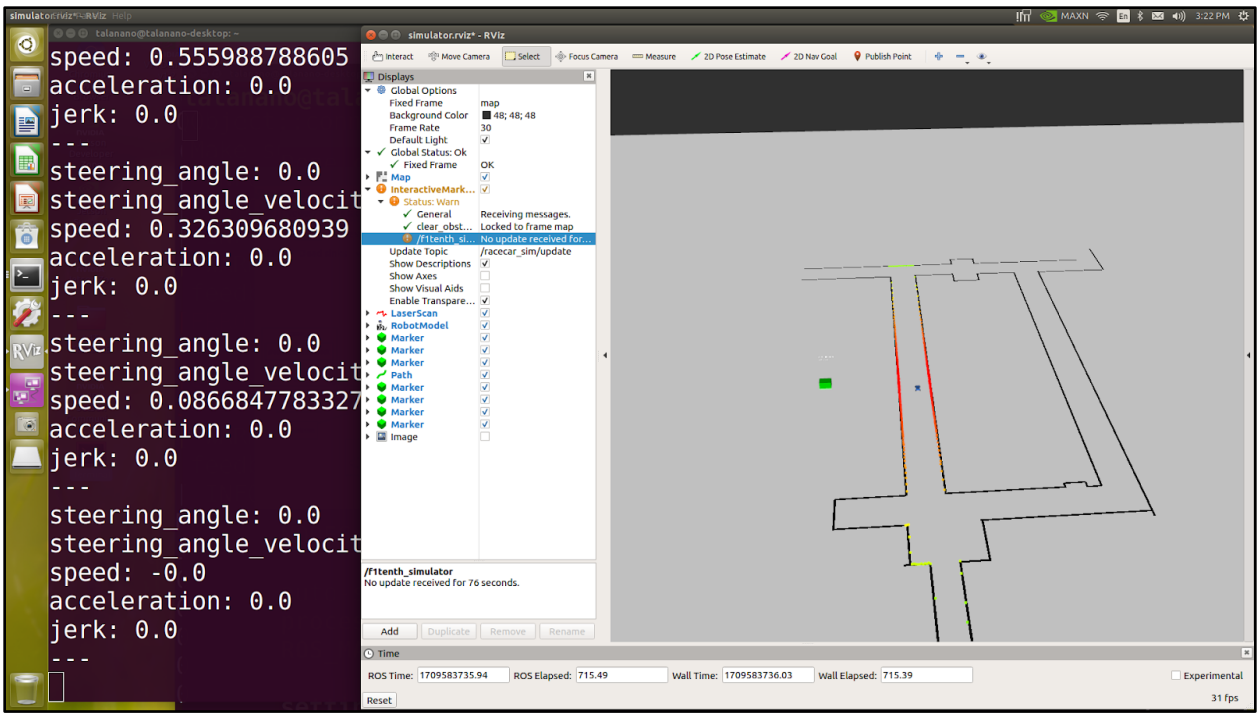

Figure 7. Updates Fields of the "/drive/drive"

Activity 16

In activity 16, the command "rostopic type /map" checks for the data type published in the "/map" topic in ROS. When we run this command, it reveals the data type to be "nav_msgs/OccupancyGrid." The mapped data is stored in a 2D occupancy grid in row-major order within the "data" array. The x and y values increment by cell resolution with increasing column and row indices, respectively. This message type is commonly used to represent occupancy grid maps, which are widely used in robotic mapping and navigation tasks.

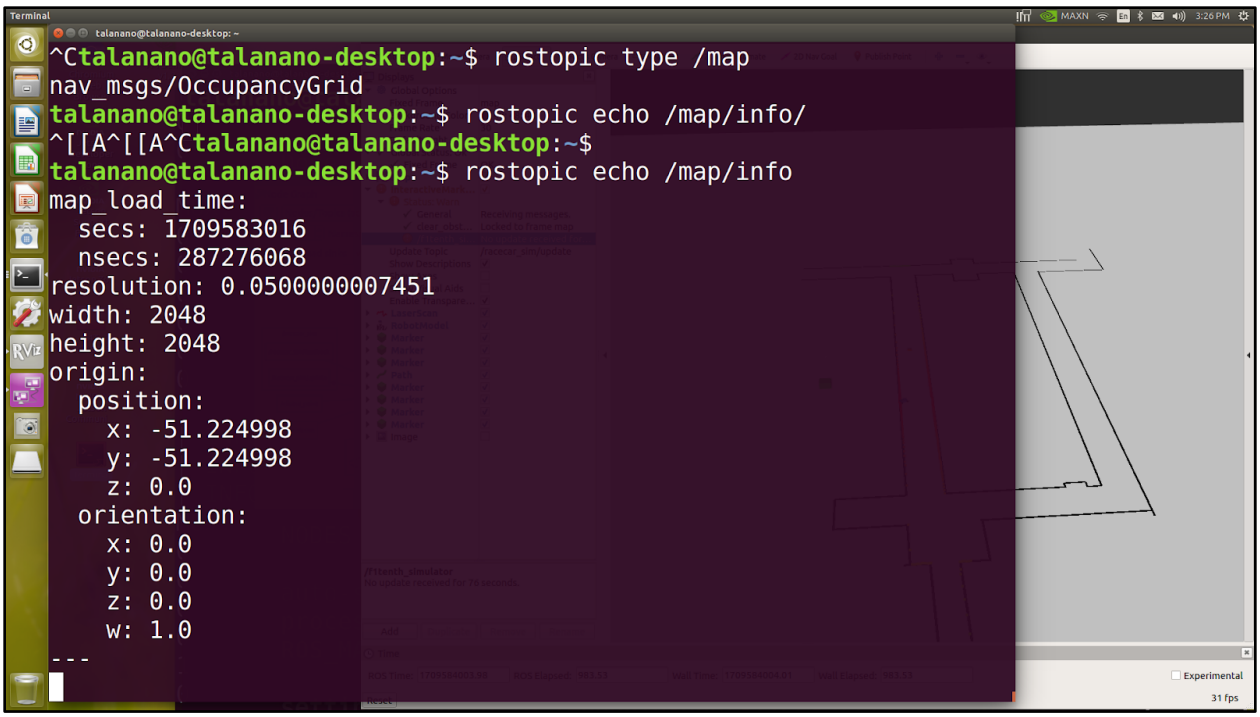

Figure 8. Mapped data in 2D grid

Activity 17

In activity 17, we examined the MapMetaData information that was published by the "map_server" node on the topic "/map". To do this, we used the "rostopic echo /map/info" command. This can be seen in Figure 8 above in the "Activity 16" section of this report. This provides us with information regarding the map load time, resolution, width, height, and origin position and orientation.

Activity 18

Various modes of motor control are speed, torque, position, and efficiency optimization.

Speed control mode ensures that the motor's speed is regulated to maintain a specific velocity setpoint. It is commonly used in EVs to ensure smooth acceleration and deceleration and maintain consistent speed on inclines and flat surfaces. In manual mode, it allows the driver to set and maintain a desired speed easily. In self-driving mode, it enables the vehicle to follow a predefined speed and react to changes in traffic conditions.

Torque control mode regulates the motor's torque output, which allows for precise control over the vehicle's acceleration and delivering power. It is suitable for EVs when high torque is required such as during steep inclines or during acceleration. In manual mode, it is very

responsive and dynamic, allowing the driver to feel more connected to the vehicle's performance. In self-driving mode, it enables efficient energy management and optimized vehicle dynamics.

Position control mode regulates the motor's position or displacement, which allows for precise control over the vehicle's movement. It is less common in EVs for primary propulsion, but is used in auxiliary systems such as steering and braking. In manual mode, it enhances maneuverability and stability, especially in tight spaces or during parking. In self-driving mode, it enables accurate trajectory tracking and path following, which is needed for navigating and obstacle avoidance.

Activity 19

In activity 19, we modified the existing simulator software stake. To do this, we added and built a new node to implement the functionality of the “Experiment” block. We began by copying the “experiment.cpp” source code file to the “~catkin_ws/src/f1tenth_simulator/node” directory. Then, we added the “<run_depend>vesc_driver</run_depend>” line to the file “~catkin_ws/src/f1tenth_simulator/package.xml.” Then, we switched to the root catkin workspace directory to rebuild the simulator package using the “catkin_make” command. This builds the executable for the new node. Finally, we added the “experiment.launch” file to the launch directory of the simulator package.

Activity 20

In Activity 20, we open the file “experiment.cpp” and examined its content. The “publish_driver_command” function is responsible for managing the transition of the desired steering angle and speed, ensuring smooth adjustments before publication. It incorporates a mechanism to regulate the rate of change in steering angle (desired_delta) and motor speed (desired_rpm), thereby preventing abrupt movements that could potentially damage mechanical components and complicate control. This smoothing and constraint mechanism promote a more gradual and controlled acceleration and turning behavior for the vehicle.

Function:

```
void publish_driver_command( const ros::TimerEvent&){
        double desired_delta = desired_steer_ang-last_servo;
double clipped_delta = std::max(std::min(desired_delta,
max_delta_servo), -max_delta_servo);
        double smoothed_servo = last_servo + clipped_delta;
last_servo = smoothed_servo; servo_msg.data=smoothed_servo;
        double desired_rpm = desired_speed-last_rpm;
        double clipped_rpm = std::max(std::min(desired_rpm, max_delta_rpm),
max_delta_rpm);
```

```
        double smoothed_rpm = last_rpm + clipped_rpm;
last_rpm = smoothed_rpm;              erpm_msg.data=smoothed_rpm;
        servo_pub.publish(servo_msg);
        erpm_pub.publish(erpm_msg);
}
```

Activity 21

In activity 21, the function “laser_callback” receives a laser scan message as the input, swaps the order of the scan data, adjusts some parameters by adding π, and then publishes the modified laser scan message. Specifically, it divides the received scan into two halves, then reverses the order of both halves, and creates a modified scan message after the angle has been adjusted before publishing. This modification is necessary to align the laser scan data with the orientation of the sensor or to meet specific requirements of the application.

Function:

“void laser_callback(const sensor_msgs::LaserScan & msg) {”:
This line defines a function named “laser_callback” that takes a “sensor_msgs::LaserScan” message as input.

“int n1=msg.ranges.size()/2;”: Calculates “n1”, which is half the size of the “ranges” array in the input message “msg”

std::vector<float> scan_ranges=msg.ranges;

“std::vector<float> scan_intensities=msg.intensities;”: Create copies of the “ranges” and “intensities” arrays from the input message “msg”, storing them in “scan_ranges” and “scan_intensities” vectors.

for (int i=0;i<n1;i++){ scan_ranges[i]=msg.ranges[n1+i];
scan_ranges[n1+i]=msg.ranges[i];
scan_intensities[i]=msg.intensities[n1+i];

“scan_intensities[n1+i]=msg.intensities[i];
}”: This loop swaps the first “n1” elements with the second “n1” elements in both “scan_ranges” and “scan_intensities”. This effectively reverses the order of the data received from the laser scan, which can be useful in some applications.

// Publish lidar information
“sensor_msgs::LaserScan scan_msg;”: Prepares new message with the modified data

“scan_msg.header.stamp = msg.header.stamp; “: Set the header information to match the input message

scan_msg.header.frame_id = msg.header.frame_id;

“scan_msg.angle_min = msg.angle_min+3.141592;”: Adjust the angles
scan_msg.angle_max = msg.angle_max+3.141592;

scan_msg.angle_increment = msg.angle_increment;
scan_msg.range_max = msg.range_max;
scan_msg.ranges = scan_ranges;
scan_msg.intensities = scan_intensities;
“lidar_pub.publish(scan_msg);”: Published the “scan_msg” using “lidar_pub” publisher
}

Activity 22

In activity 22, we began to manually drive the vehicle using the joystick. We opened the “params.yaml” file and viewed the “max_speed” parameter value. We ensured that this value was between 1 to 1.5m/s as this range limits the vehicle speed command to a safe maximum value. We also ensured that the “max_steering_angle” was set to 0.41 radians and the joystick was put into “x” mode.

Activity 23

In activity 23, we ran the “postlaunch f1tenth_simulator experiment.launch” command to start operating the vehicle. This initiates the operation of the vehicle in the F1TENTH Simulator virtual environment.

Activity 24

In activity 24, we ran the “rosnode list” command to find the running nodes in the system. By running this command, we are able to get a list of the nodes in the system that are running at the time.

Activity 25

The nodes identified in the “rosnode list” command are running processes that are responsible for performing different functions within the ROS environment.

“/behavior_controller”: This controls the behavior of the vehicle. It determines the appropriate actions to take based on inputs from sensors and other sources (navigation goals).

“/f1tenth_simulator”: This simulates F1TENTH racecar and its environment by providing a virtual platform for testing and developing algorithms related to navigation, control, and perception.

“/joy_node”: This node interfaces with the joystick and publishes the inputs from the controller as ROS messages. It allows users to manually control the vehicle using a joystick.

“/map_server”: This node loads and publishes maps of the vehicle’s environment. Such map data are used for localization, path planning, and navigation to other nodes in the system.

“/racecar/robot_state_publisher”: This publishes the state of the robot, including its position, orientation, and velocity, as a transform message. It displays robot’s state in “RViz” virtual environment.

“/random_walker”: This implements a random walking behavior for the robot. It generates random movements or actions to explore its environment without a predefined plan.

“/rosout”: This captures log messages generated by other nodes in the system and publishes them to the “/rosout” topic. It is used for debugging and monitoring the performance of the ROS system.

“/rqt_gui_py_node_12113”: This is a node associated with an RQT GUI plugin, which provides a graphical user interface for interacting with the ROS system.

“/rviz”: This is a visualization tool that allows users to visualize and analyze data from ROS topics in a 3D environment.

“/vesc_driver_node”: This node interfaces with the VESC and sends commands to control the vehicle's speed and steering.

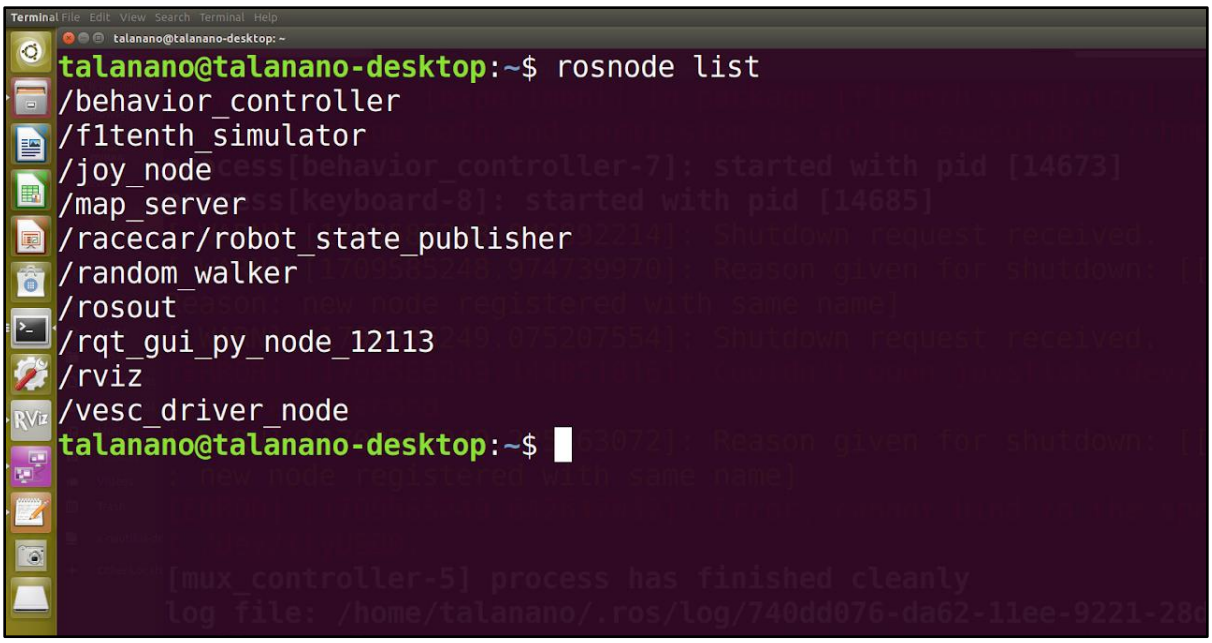

Figure 9. "rosnode list" Command Output

Activity 26

In activity 26, we viewed the list of topics published over the ROS environment. To do this, we executed the “rostopic list” command. Figure 10 below displays all the listed topics that were outputted after running the “rostopic list” command.

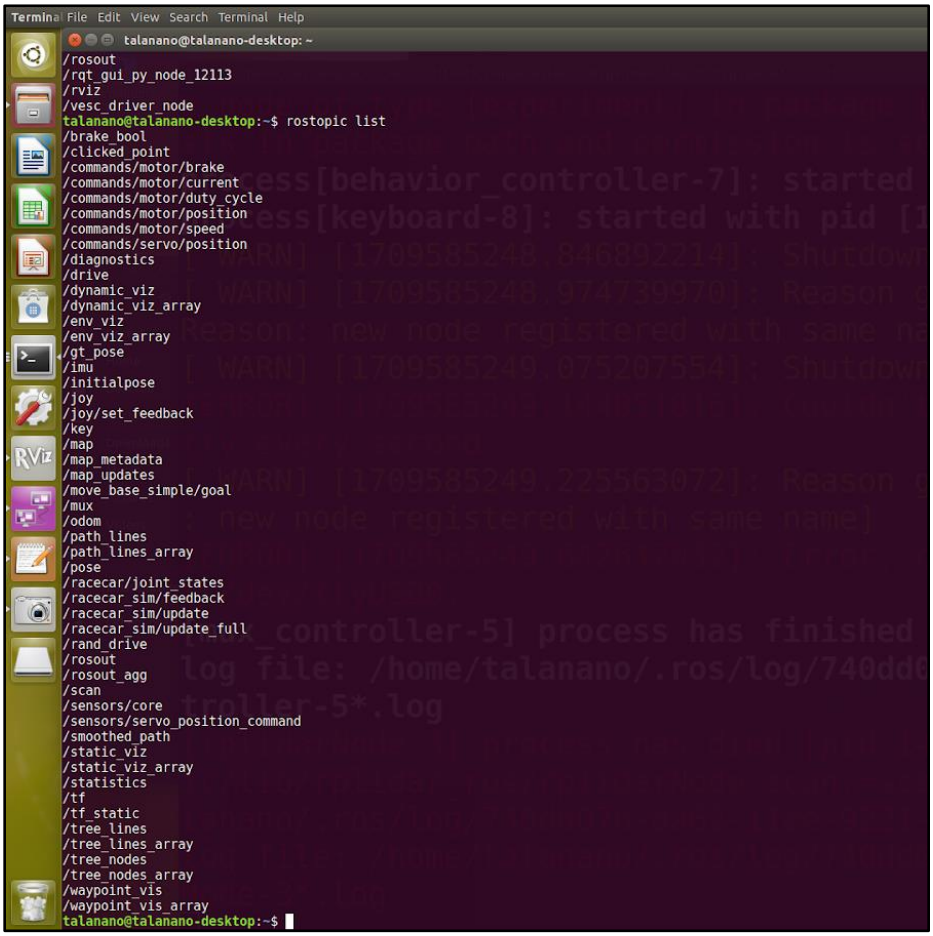

Figure 10. “rostopic list” Command Output

Activity 27

The ROS topics associated with VESC ROS driver are shown above in Figure 10. The rate at which the motor speed and servo steering angle commands are published commands that are related to the vehicle's electronic speed controller. Such topics are commands for motor speed and steering angle.

The rate of execution for the "/vesc_driver_node" and the frequency of its publication commands to its associated topics, may be affected by factors like computational workload, the complexity of the commands, and the hardware's responsiveness.

Activity 28

In activity 28, we pressed and released the LB button on the joystick to enable manual driving. We used the left mini-stick to send steering commands to the vehicle, and the right mini-stick to send speed commands to the vehicle. After completing the drive, we shut down the program using ctrl-c in the command window.

Activity 29

Derivation of $k_\omega = \frac{15pg_r}{\pi r_\omega}$:

$$\omega_e = \frac{2\pi f}{p} g_r$$

$$v_s = k_\omega \omega_e r_\omega$$

$$v_s = k_\omega \left(\frac{2\pi f}{p} g_r\right) r_\omega$$

$$k_\omega = \frac{v_s}{\frac{2\pi \left(\frac{RPM}{60}\right) g_r r_\omega}{p}}$$

$$k_\omega = \frac{15pg_r}{\pi r_\omega}$$

Therefore, $k_\omega = \frac{15pg_r}{\pi r_\omega}$.

Activity 30

In activity 30, we fine tuned the offset value δ_0. By driving the vehicle in a straight line without providing any steering command, we observed to see if the vehicle drifted to the left or right. Based on this, we made a small adjustment to δ_0 based on the drift direction. This process was repeated until the vehicle stopped drifting away from the straight path.

Activity 31

In activity 31, we began by placing the vehicle at the starting position. Then we provided the maximum steering command to the vehicle using the mini-stick and pushing it all the way in the turn direction. We observed how the vehicle makes the turn and crosses over the line again. By marking the position where the centre of the rear axle crosses the line, we can observe the distance between the start and end points to obtain the value of $2r$. To obtain the desired steering angle value, δ_d, we get this from the corresponding field in the published topic "/drive."

$2r = 1.34m$

$r = 0.67m$

$Steering\ angle, \delta_d = 0.57320001468$

Activity 32

In activity 32, we used the results of the calibration experiment to obtain the value of k_δ. Then, we updated the value of this parameter in the "params.yaml" file.

Activity 33

In the "experiment.cpp" file, a timer is set up in ROS using "createTimer" to execute the "publish_driver_command" function of the "RacecarExperiment" class to regular intervals, with the frequency controlled by "driver_smoother_rate". This timer guarantees that commands for controlling the vehicle's speed and steering are published consistently, leading to smoother operation of the vehicle's actuators.

Activity 34

In activity 34, we executed the “rostopic hz /commands/motor/speed” command. This command measures and displays the publishing rate of messages on the "/commands/motor/speed" topic in ROS. It provides information about how frequently messages are being published to this topic, indicating the rate at which speed commands are being sent to control the motors. This information can be useful for monitoring the activity and performance of the motor control system in real-time.

Activity 35

“rosrun”: This command runs a single node directly. It is useful for testing single components.

“roslaunch”: This command launches multiple nodes with customized configurations from a single launch file. This is a convenient method for initiating intricate systems.

“rosnode”: This command lists and manages active nodes. This is a useful practice for debugging system behavior effectively.

“rostopic”: This command interacts with ROS topics and allows for tasks such as listing topics, obtaining their types, checking message frequencies, and inspecting message contents, which are all crucial for troubleshooting and data inspection purposes.